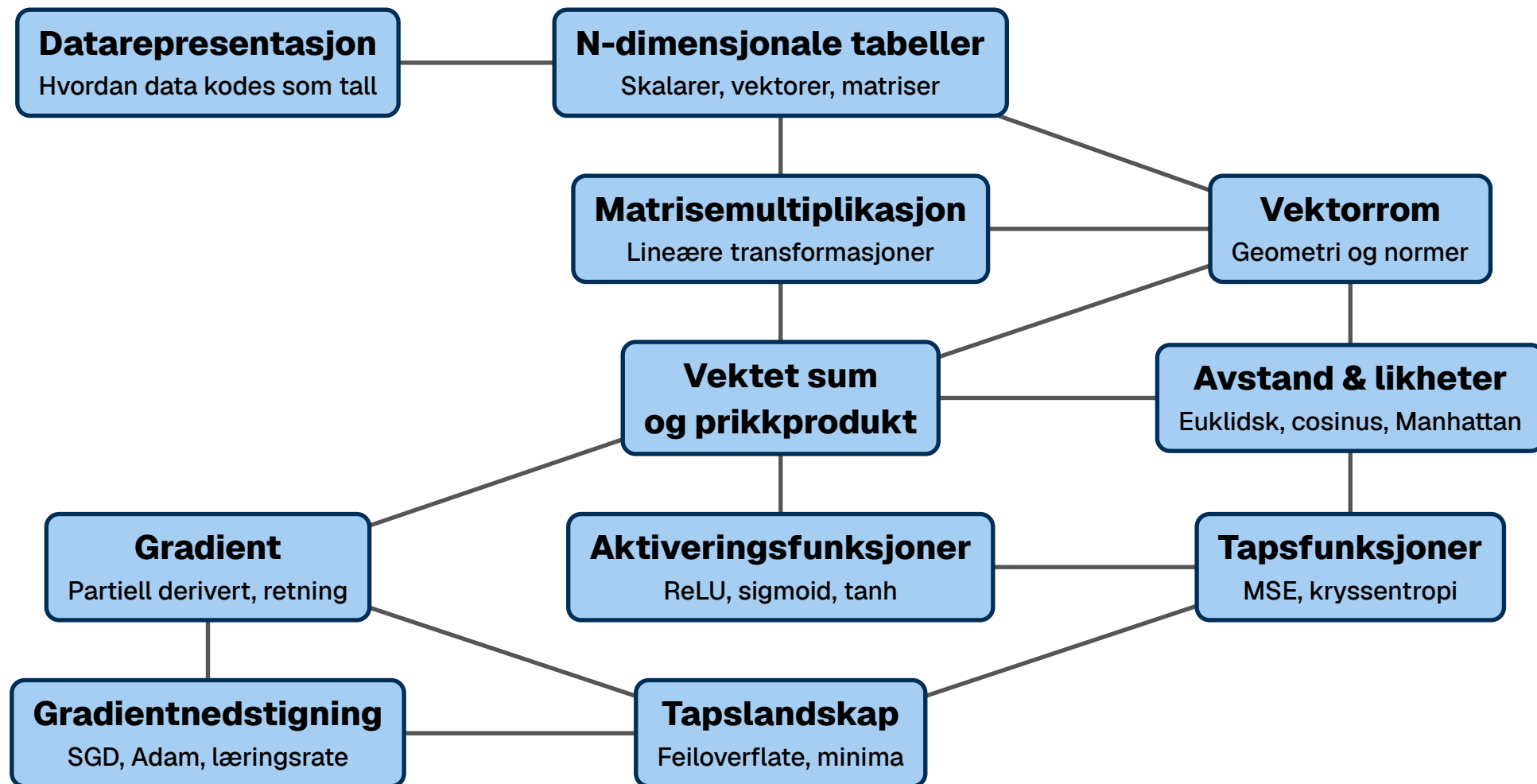


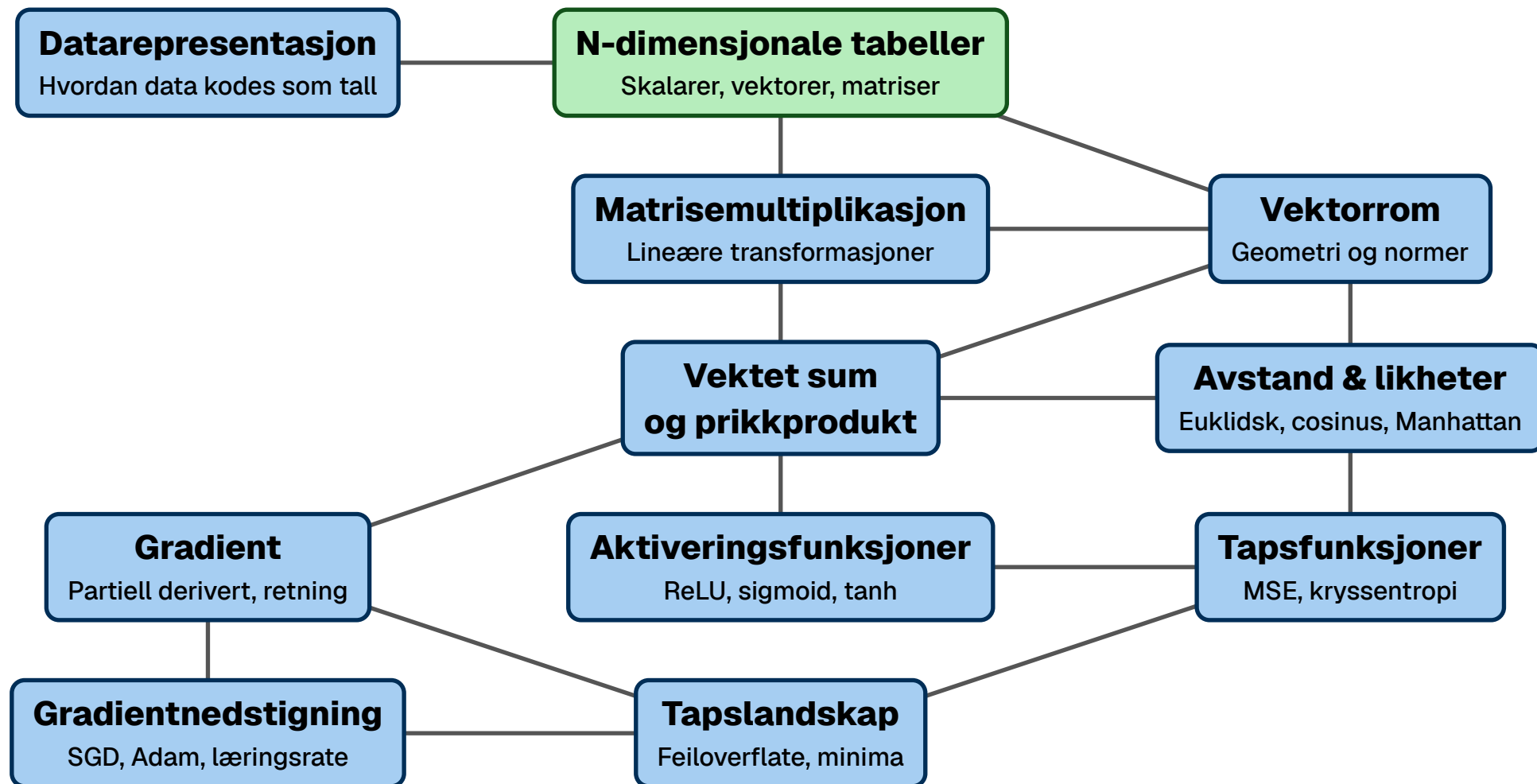
REPETISJON: MATEMATIKKEN I MASKINLÆRING

18.05.2026

HVORDAN DE MATEMATISKE BYGGESTEINENE HENGER SAMMEN I MASKINLÆRING:



N-DIMENSJONALE TABELLER



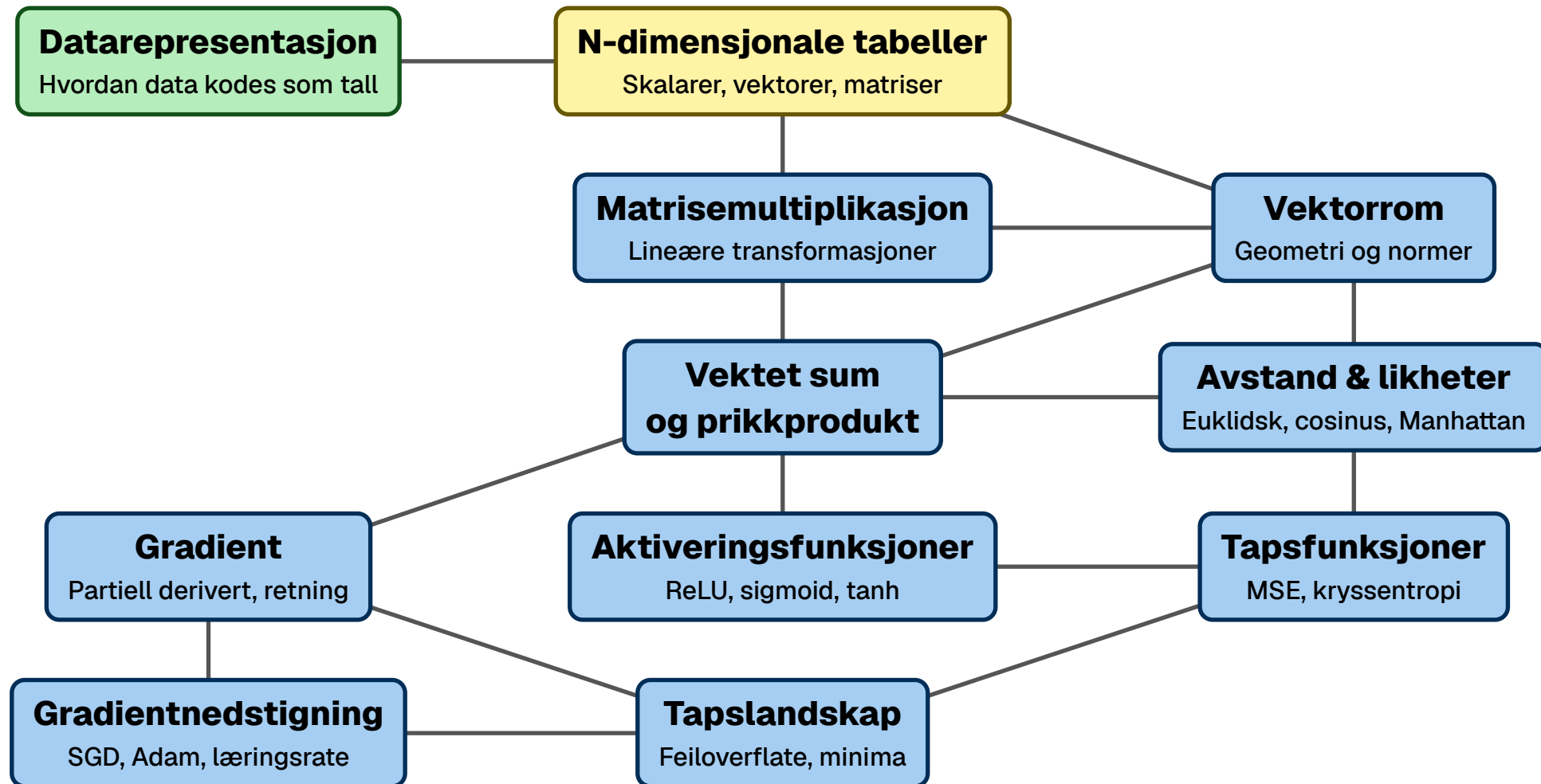
BEHOLDERE FOR NUMERISK DATA

Alt i maskinlæring starter med **tall** — og vi trenger gode måter å organisere dem på.

- **Skalar x** : Et enkelt tall, f.eks. én måleverdi eller én parameter ($x \in \mathbb{R}$).
- **Vektor x** : En ordnet liste med tall. Representerer ett datapunkt i et flerdimensjonalt rom.
- **Matrise W** : En 2D-tabell med tall. Brukes til å transformere data eller holde på vekter.
- **Tensor**: Fellesbetegnelse i ML-miljøet for skalarer, vektorer, matriser og høyere dimensjoner.
 - **0-D (skalar)**: Et enkelt tall (f.eks. -3.56)
 - **1-D (vektor)**: En ordnet liste (representerer ett datapunkt).
 - **2-D (matrise)**: En tabell med rader og kolonner.
 - **3-D+ (N-dimensjonal array)**: F.eks. en video: $\mathbf{X} \in \mathbb{R}^{T \times H \times W \times C}$ med T frames, høyde H , bredde W og $C = 3$ RGB-kanaler.

Merk: *Tensor* er et lånt begrep — det har en annen, mer presis betydning i fysikk og matematikk.

DATA REPRESENTASJON



REPRESENTERE DATA SOM TALL

Nå som vi vet hva en vektor er – hvordan fyller vi den med meningsfullt innhold?

- Datamaskiner forstår bare tall — alt rå data må konverteres til **numerisk form**
- Kategoriske verdier (f.eks. «katt», «hund») → heltall eller one-hot-vektorer
- Bilder → matriser av pikselintensiteter (0–255)
- Tekst → tallvektorer via tokenisering og embedding

Eksempel: Hvis vi skal forutsi boligpriser, kan én bolig representeres som en **vektor** der komponentene er areal, antall rom og byggeår: $x = (85, 3, 1998)$.

HVA ER ET TREKK?

En vektor er bare nyttig hvis komponentene representerer noe meningsfullt. Det kaller vi **trekk (features)**.

- **Observerbare egenskaper:** Målbare attributter ved dataene som er relevante for oppgaven.
- **Numeriske trekk:** Kontinuerlige verdier som høyde (cm) eller temperatur (°C).
- **Binære trekk:** Ja/Nei representert som 1 eller 0 (f.eks. «fagorganisert?»).
- **Kategoriske trekk:** Yrke, farge, ordtype — konverteres til numerisk form før bruk.

Valg av trekk er en av de viktigste beslutningene i et ML-prosjekt. Gode trekk gjør modellen enklere; dårlige trekk gjør den ubrukelig uansett hvor sofistikert algoritmen er.

Oppgave: finn ett dårlig og ett bra trekk som beskriver fugler.

REPRESENTERE SPRÅK

En spesielt elegant strategi for å representere **tekst** som tall bygger på en enkel idé:

Ord som forekommer i liknende kontekster har liknende betydning.

- **Vokabular:** Definer ordene vi ønsker å modellere som mål-vektorer.
- **Kontekst:** Definer nabolaget (f.eks. ± 5 ord eller ord i samme setning).
- **Telling:** Tell samforekomster for å bygge trekkvektorene.

Resultat: Semantisk likhet – “hund” og “valp” betyr noe lignende så deres vektorer bør også være ganske like (hva det betyr kommer vi til).

REPRESENTERE SPRÅK

I praksis går vi gjennom flere rensetrinn før vi teller:

1. **Tokenisering:** Splitter tekst til setninger og ord.
2. **Normalisering:** Konverterer til små bokstaver og fjerner tegnsetting.
3. **Stoppliste:** Filtrerer ut høyfrekvente funksjonsord (som «og», «i», «det»).
4. **Frekvensgrense (Zipfs lov):** Beholder kun ord med frekvens $f \geq 10$ for å unngå støy.

Hvorfor trenger vi en frekvensgrense? Fordi ordfrekvenser følger en ekstremt skjev fordeling.

- Noen få ord dominerer frekvensen fullstendig (“som”, “og”, “i”, “det”).
- De fleste ordtypene i vokabularet forekommer svært sjelden.
- Frekvensen er omtrent omvendt proporsjonal med rangeringen:

$$f \approx \frac{1}{r}$$

Dette betyr at ord nummer 2 er omtrent halvparten så vanlig som ord nummer 1, ord nummer 10 er en tidel så vanlig, osv. Sjeldne ord gir lite informasjon — derav frekvensgrensen.

TELLEVEKTOR OG EMBEDDINGS

Med disse rensetrinnene kan vi nå bygge to typer vektorrepresentasjoner:

Tellebaserte vektorer

- **Høydimensjonale:** $d = |V|$ (like mange dimensjoner som ord i vokabularet)
- **Sparse:** De fleste verdier er null — de fleste ord forekommer ikke i hvert dokument.
- **Diskret:** Hvert trekk svarer til ett spesifikt ord.

Embeddings (lærte representasjoner)

- **Lavdimensjonale:** $d = 200\text{--}300$
- **Dense:** Fyllt med reelle tall — ingen null-verdier.
- **Distribuert:** Informasjon er kodet i **helheten** av vektoren, ikke i én komponent.

DOKUMENT-TERM-MATRISE

Den enkleste formen for tellebasert representasjon er **Bag-of-Words (BoW)**. Vi lager en matrise der radene er dokumenter og kolonnene er ord:

Dokument	battle	good	fool	wit
As You Like It	1	114	36	20
Twelfth Night	0	80	58	15
Julius Caesar	7	62	1	2
Henry V	13	89	4	3

BoW ignorerer rekkefølge og grammatikk, men fanger det semantiske innholdet via ordvalg. “Julius Caesar” skiller seg klart fra “As You Like It” — bare ved å telle ord.

TF-IDF VEKTING

Et problem med rå tellevektorer: vanlige ord som “good” dominerer alle dokumenter og sier lite om hva som **skiller** dem. Løsningen er **TF-IDF**:

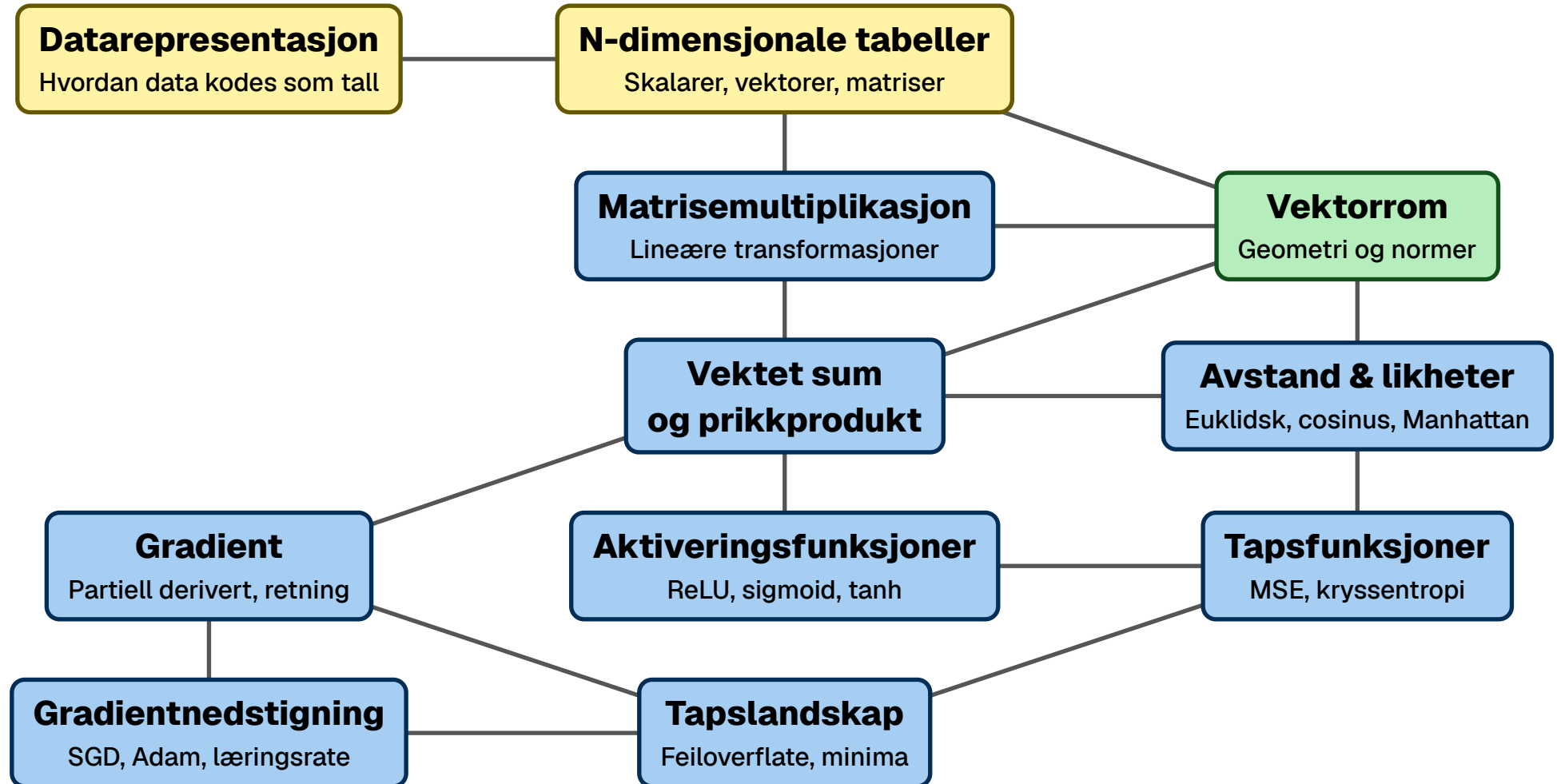
- **TF (Term Frequency)**: Hvor ofte ordet t_i forekommer i dokument d_j .
- **IDF (Inverse Document Frequency)**: Nedveker ord som forekommer i nesten alle dokumenter.

$$\text{tf-idf}(t_i, d_j) = \text{tf}(t_i, d_j) \times \log \left(\frac{\text{antall dokumenter}}{\text{antall dokumenter som har ordet } t_i} \right)$$

En høy vekt oppnås når en term er vanlig i ett spesifikt dokument, men sjelden i samlingen totalt — nøyaktig det som gjør et ord **karakteristisk**.

Intuisjon: Ordet “battle” i “Henry V” er mer informativt enn “good” fordi det forekommer sjelden på tvers av alle Shakespeare-stykker. TF-IDF belønner slik særegenhet.

VEKTORROM



HVOR DATAEN BOR

Vi har nå vektorer fulle av tall. Men hva **betyr** egentlig en vektor geometrisk?

- En vektor er ikke bare en liste med tall — den eksisterer i et **vektorrom**.
- Et vektorrom $\mathbb{R}^n$ er et rom med n dimensjoner — én per trekk i datasettet.
- Hvert datapunkt er ett **punkt** i dette rommet.
- Geometriske begreper som **avstand**, **retning** og **projeksjon** gir mening i et slikt rom.
- Lineære modeller “kutter” vektorrommet med hyperplaner for å klassifisere data.

Eksempel: Med trekkene “areal” og “pris” bor hvert hus i $\mathbb{R}^2$ — et vanlig 2D-plan. Med 100 trekk bor det i $\mathbb{R}^{100}$. Vi kan ikke visualisere det, men matematikken fungerer nøyaktig likt.

GEOMETRISK REPRESENTASJON

Som nevnt fungerer hver trekktype som en **dimensjon** — en akse — i et koordinatsystem.

- Objektene vi representerer blir **punkter** i dette rommet.
- To objekter som ligner hverandre burde ligge **nær hverandre** i rommet.
- Dette er den grunnleggende intuisjonen bak nesten all maskinlæring.

Tenk tilbake på BoW-matrisen: hvert Shakespeare-stykke er et punkt i et ord-rom. “Henry V” og “Julius Caesar” ligner hverandre mer enn de ligner “As You Like It” — og det bør gjenspeiles i **avstand**. Vektorene for “hund” og “valp” bør også ligge nærme hverandre i vektorrommet, dette gjelder for både tellevektorer og embeddings.

DIMENSJONALITET

- $d = 2$: Et vanlig plan — lett å visualisere.
- $d = 3$: Et rom vi kan forestille oss.
- $d > 1000$: Høydimensjonalt — vanskelig å forestille seg, men matematisk veldefinert.

Høydimensjonale representasjoner med tusenvis av trekk er vanlige i moderne maskinlæring. Vokabularstørrelsen i språkmodeller kan være 50 000 ord eller mer.

PUNKTER VS. VEKTORER

Det er nyttig å skille mellom to perspektiver på det samme objektet:

Som et punkt: En posisjon i rommet — hvor er dette datapunktet?

Som en vektor: En retning og lengde fra origo — hvilke egenskaper har dette datapunktet?

Vektornotasjon og norm:

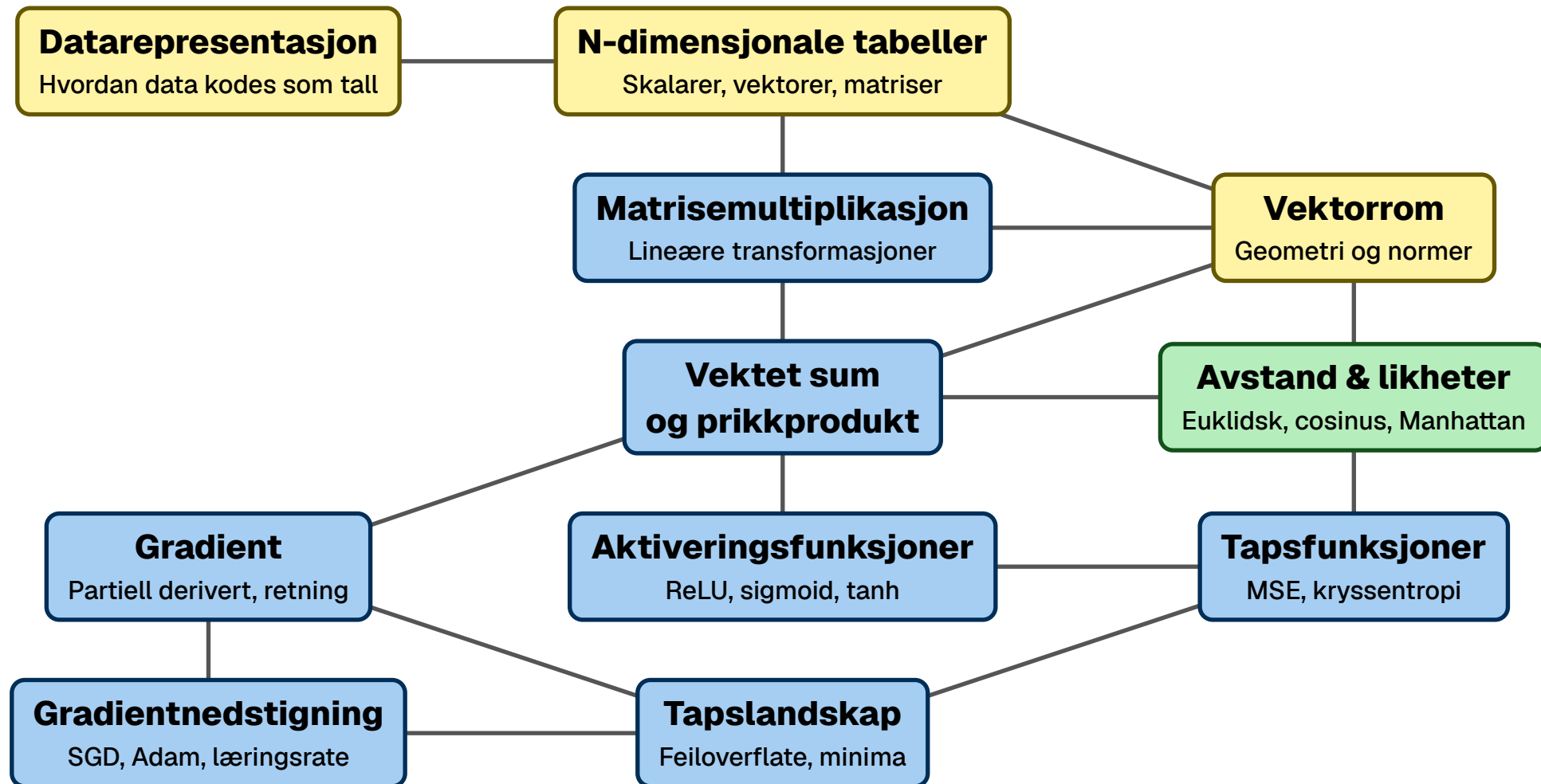
$$\mathbf{v} = [v_1, v_2, \dots, v_d]$$

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^d v_i^2}$$

Normen $\|\mathbf{v}\|$ er vektorens **lengde** — avstanden fra origo til punktet. Den gir oss et første mål på “størrelse”.

Oppgave: Finn normen til $[1, 2, 3]$ og $[0.577, 0.577, 0.577]$

AVSTAND OG LIKHETER



MÅLE LIKHET I ROMMET

Nå som datapunktene bor i et vektorrom, kan vi stille det viktigste spørsmålet: **hvor like er to punkter?**

- For å sammenligne to datapunkter (vektorer), trenger vi et mål på **avstand** mellom dem.
- **Normen** $\|x\|$ måler lengden til én vektor — grunnlaget for å beregne distance mellom to.

Euklidsk distanse (L2-distanse):

$$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- Geometrisk: den “rett-linje”-avstanden mellom to punkter i rommet.
- Jo kortere avstand, jo likere er datapunktene i modellen vår.

Embeddings revisited: Husk de lærte representasjonene vi nevnte tidligere. To ord med liknende betydning skal ha lav euklidisk distanse mellom seg — tuppen av vektorene ligger nærme hverandre.

COSINUS-LIKHET

Euklidsk avstand tar hensyn til **størrelse**. Men noen ganger bryr vi oss mer om **retning**.

To dokumenter der det ene er dobbelt så langt som det andre, men inneholder de samme ordene i samme proporsjoner, bør anses som like — selv om den euklidske avstanden er stor.

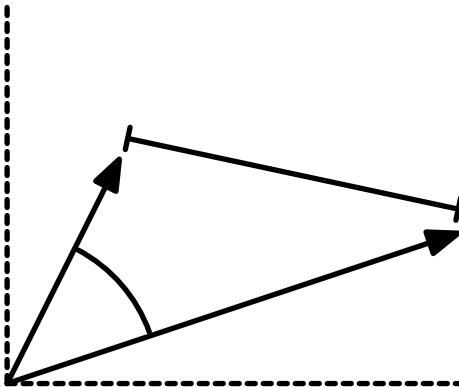
Cosinus-likhet måler vinkelen mellom to vektorer:

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \in [-1, 1]$$

- $\cos(\theta) = 1$: Vektorene peker i nøyaktig samme retning — identiske.
- $\cos(\theta) = 0$: Vektorene er ortogonale — ingen likhet.
- $\cos(\theta) = -1$: Vektorene peker i motsatt retning.

Embeddings revisited: Husk de lærte representasjonene vi nevnte tidligere. To ord med liknende betydning skal ha høy cosinus-likhet — de peker i samme retning i embedding-rommet.

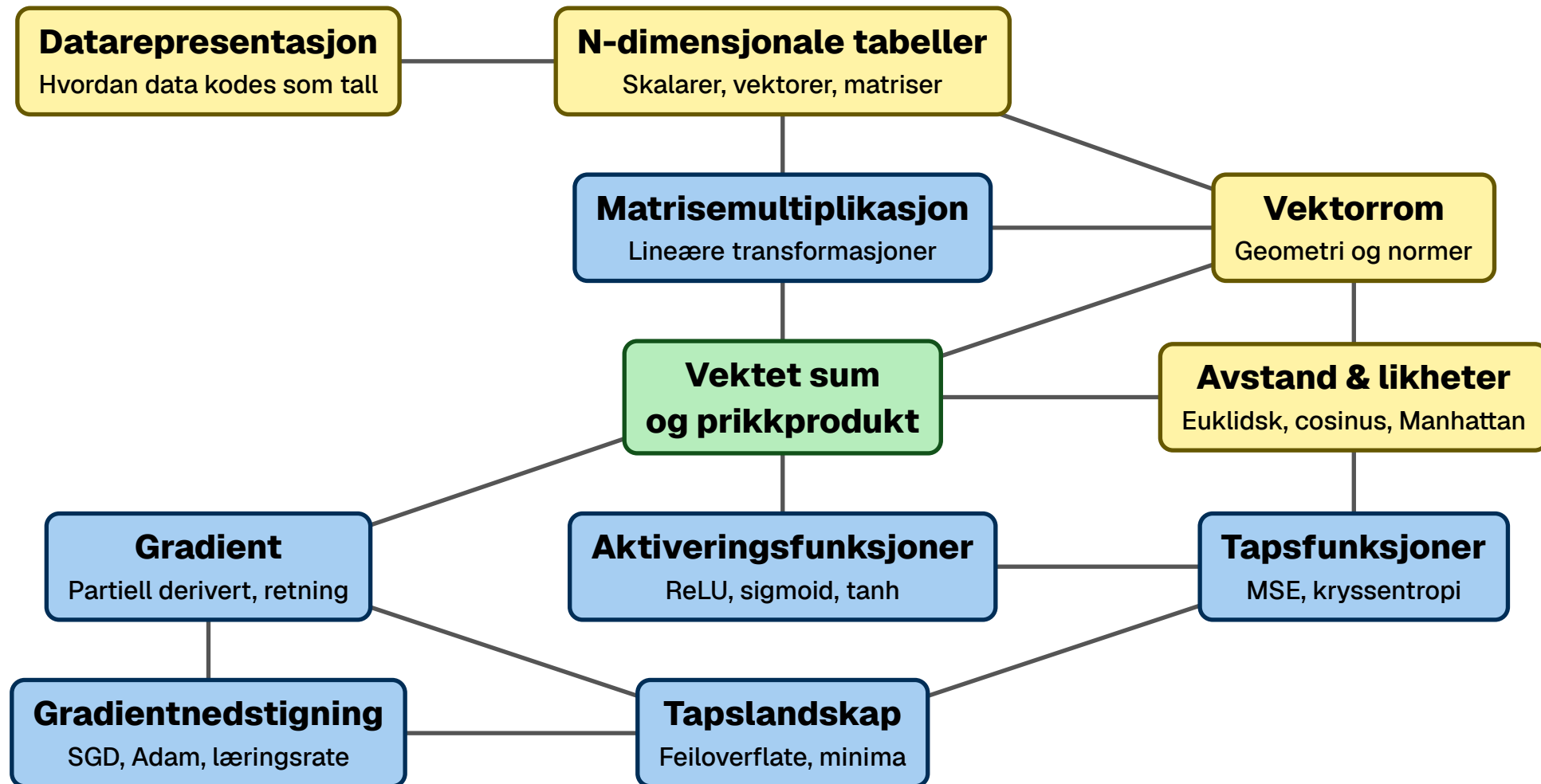
COSINUS VS EUKLIDISK



Oppgave: Hva er fordeler og ulemper med metodene?

Oppgave: Hva kan vi gjøre for at euklidisk avstand og cosinus-likhet oppfører seg mer likt?

VEKTET SUM OG PRIKKPRODUKT



PRIKKPRODUKTET — HJERTE I MASKINLÆRING

Legg merke til at cosinus-likheten inneholder et uttrykk vi ikke har sett på enda: $x \cdot y$.

Dette kalles **prikkproduktet**, og det er kjerneoperasjonen i nesten all maskinlæring.

Prikkprodukt mellom w og x :

$$w \cdot x = \sum_{i=1}^n w_i x_i = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

- w er **vektene** (hva modellen har lært), x er **inputtrekkene**.
- Resultatet er ett enkelt tall — en **score** eller **prediksjon**.
- Dette er matematisk identisk med en **vektet sum** — vi summerer trekkene, men lar noen telle mer enn andre.

Intuisjon: Vektene forteller modellen hvilke trekk som er **viktigst**. En høy vekt w_i betyr at trekk x_i påvirker prediksjonen mye — en lav vekt betyr at den nesten ignoreres.

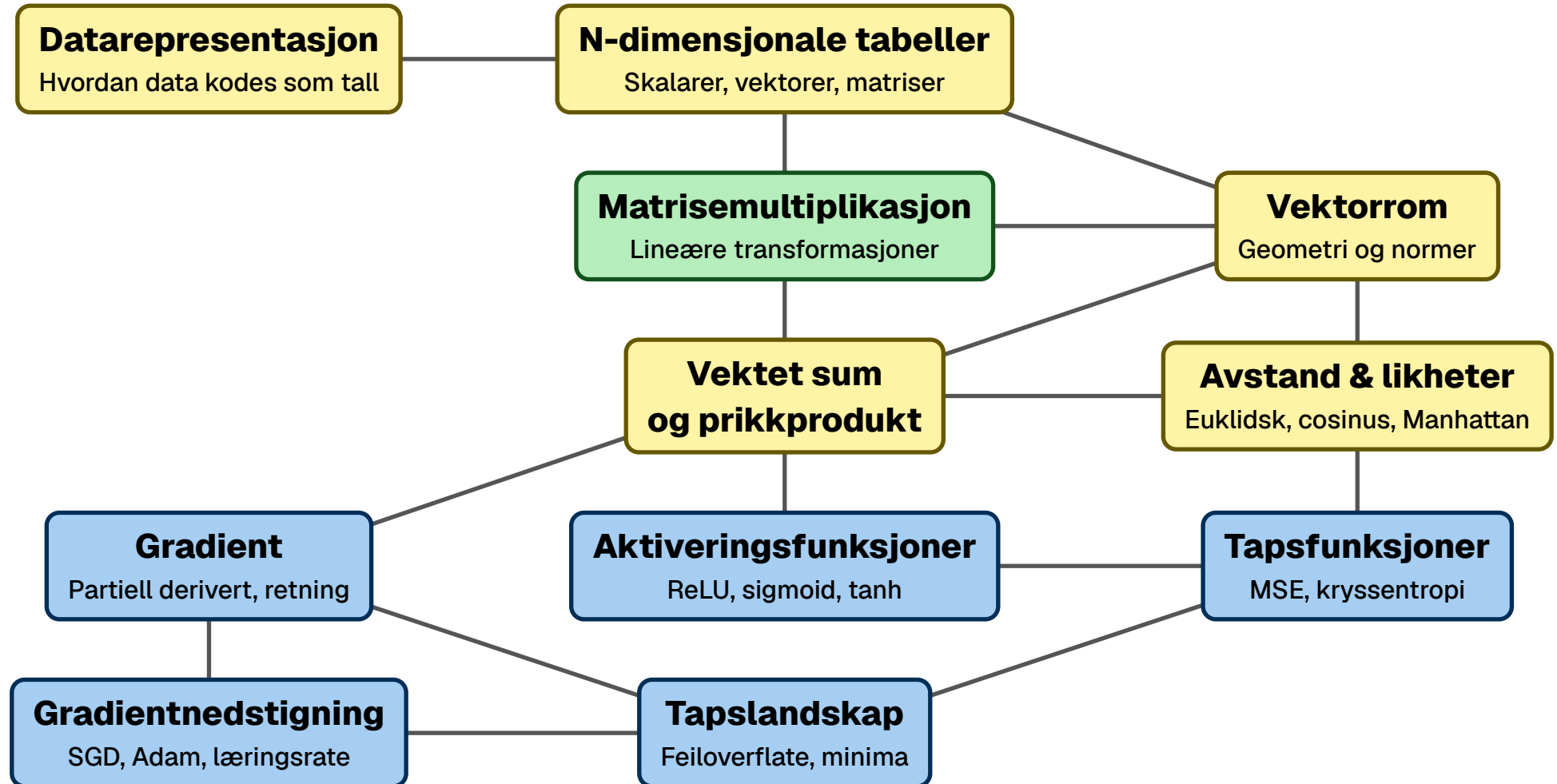
Oppgave: Regn ut $[1, 2, 0] \cdot [1, 2, 0]$.

PRIKKPRODUKTET — HJERTE I MASKINLÆRING

Prikkproduktet kan blant annet brukes til:

- Å finne lengden (normen) på en vektor $\|x\|^2 = x \cdot x$
- Å finne cosinuslikhet
- Å finne en vektet sum
- Å projisere en vektor ned på en annen

MATRISEMULTIPLIKASJON



FRA PRIKKPRODUKT TIL MATRISEMULTIPLIKASJON

Vi kan regne ut én vektet sum med ett prikkprodukt. Men hva om vi vil beregne **mange** vektete summer samtidig?

- En vektet sum er prikkproduktet mellom én vektrad og én inputvektor.
- Setter vi mange slike vektrader **oppå hverandre**, får vi en **matrise W** .
- Matrisemultiplikasjon er da mange prikkprodukter **parallelt** — én per rad i **W** .
- Moderne CPU- og GPU-arkitekturer er spesialbygde for nettopp denne operasjonen.

Matrisemultiplikasjon transformerer data til nye representasjoner effektivt.

MATRISEMULTIPLIKASJON I PRAKSIS

Gitt en vektmatrise $\mathbf{W} \in \mathbb{R}^{m \times n}$ og inputvektor $\mathbf{x} \in \mathbb{R}^n$:

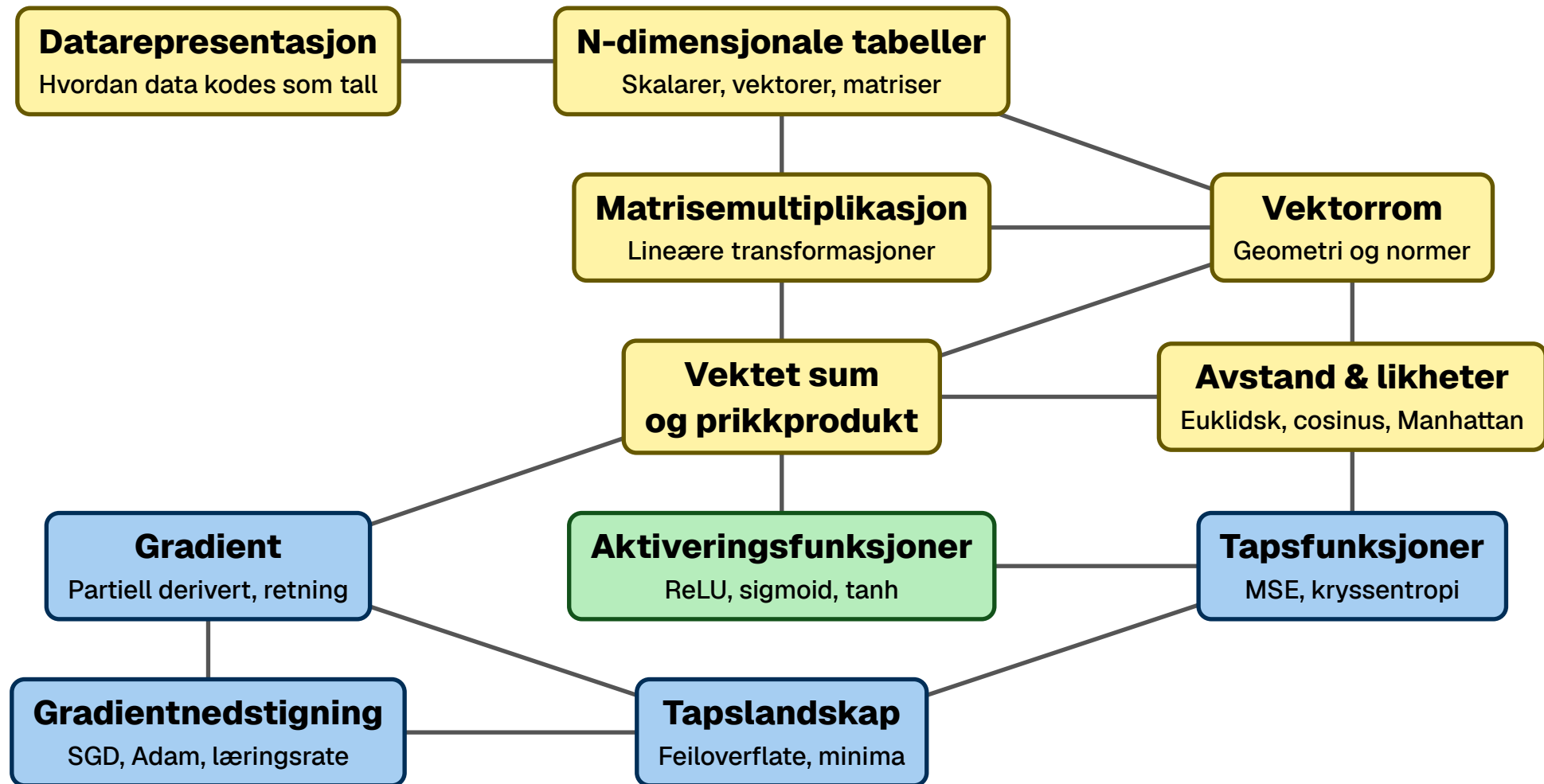
$$\mathbf{W}\mathbf{x} = \begin{pmatrix} w_{11} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m1} & \dots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \mathbf{w}_1 \cdot \mathbf{x} \\ \vdots \\ \mathbf{w}_m \cdot \mathbf{x} \end{pmatrix}$$

- Hvert element i resultatet er ett **prikkprodukt** — én vektet sum.
- Et fullt koblet (dense) lag i et nevralt nettverk er nettopp $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
- $\mathbf{b}$ er en **bias-vektor** som forskyver resultatet — som skjæringspunktet i en lineær funksjon.

Dimensjoner: $(m \times n) \cdot (n \times 1) = (m \times 1)$. Antall kolonner i $\mathbf{W}$ må matche antall rader i $\mathbf{x}$. Vi «komprimerer» eller «utvider» representasjonen avhengig av m .

Oppgave: Gi deg selv tre valgfrie vektorer $\rightarrow$ regn to prikkprodukter med vektorene $\rightarrow$ regn ut de samme prikkproduktene med en matrise og en vektor

AKTIVERINGSFUNKSJONER



PROBLEMET MED LINEARITET

Vi kan nå beregne $z = \mathbf{W}x + \mathbf{b}$. Men hva skjer om vi legger på flere lag?

To lineære transformasjoner etter hverandre er fremdeles lineær: $\mathbf{W}_2(\mathbf{W}_1x) = (\mathbf{W}_2\mathbf{W}_1)x$.

Uansett hvor mange lag vi har, kollapser alt til én enkelt lineær funksjon. Vi trenger noe mer.

AKTIVERINGSFUNKSJONER — INTRODUSERE IKKE-LINEARITET

Løsningen er å sette en ikke-lineær funksjon **etter** hvert lag. Disse kalles **aktiveringsfunksjoner**.

Sigmoid — brukes i output for binær klassifisering:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

ReLU (Rectified Linear Unit) — vanligste valg i skjulte lag:

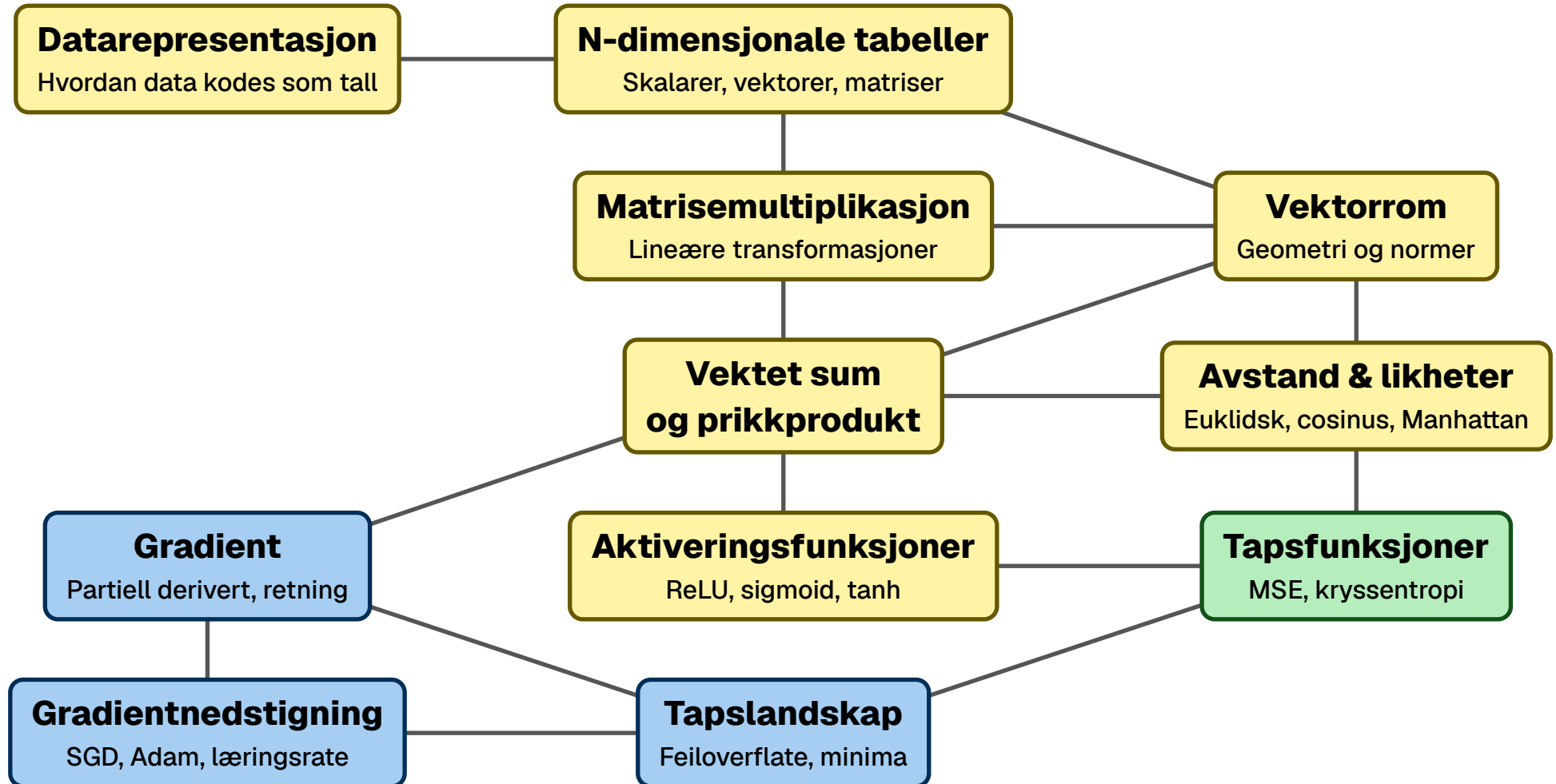
$$\text{ReLU}(z) = \max(0, z)$$

Softmax — for flerklasse-klassifisering, gir sannsynlighetsfordeling over K klasser:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Intuisjon: Sigmoid “klemmer” alle tall inn i $(0, 1)$ — nyttig når svaret er en sannsynlighet. ReLU “slukker” negative aktivasjoner og er rask å trene. Softmax fordeler total sannsynlighet (summer til 1) over alle klasser.

TAPSFUNKSJONER



FRA PREDIKSJON TIL FEILMÅLING

Nå har vi alle ingrediensene til å gjøre en prediksjon: representere data som vektorer, transformere dem med matrisemultiplikasjon, og klemme resultatet gjennom en aktiveringsfunksjon.

Men hvordan vet modellen om den **tar feil** — og **hvor mye** feil?

Det er her vi vender tilbake til geometrien fra avstandsmålingene. En **tapsfunksjon (loss function)** bruker avstandsmåling til å gi oss ett enkelt tall på hvor dårlig prediksjonen er.

$$L = \text{avstand}(\hat{y}, y)$$

En god tapsfunksjon bør: (1) være null når prediksjonen er perfekt, (2) vokse jevnt når feilen øker, og (3) være deriverbar — slik at vi kan optimere den.

MEAN SQUARED ERROR (MSE)

For **regresjon** (predikere kontinuerlige verdier) er den naturlige tapsfunksjonen direkte knyttet til den euklidske distansen vi allerede kjenner:

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Dette er kvadratet av den euklidske distansen mellom prediksjonsvektoren og fasitvektoren.

Minner om formelen for **euklidisk distanse**:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Oppgave: Hvis feilen øker lineært, hvorfor kvadratiserer vi den? Hvorfor bruker vi ikke ren euklidisk distanse?

BINÆR KRYSSSENTROPI

For **klassifisering** bruker vi sigmoid som aktivering — output er en sannsynlighet $\hat{y} \in (0, 1)$. Da passer ikke MSE like godt.

Binær kryssentropi er utledet fra informasjonsteori og straffer hardt når modellen er **svært sikker, men feil**:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Hvis $y = 1$ og $\hat{y} \rightarrow 0$: tapet $-\log(\hat{y}) \rightarrow \infty$ — katastrofalt.
- Hvis $y = 1$ og $\hat{y} \rightarrow 1$: tapet $-\log(\hat{y}) \rightarrow 0$ — perfekt.

Sammenheng: Når vi gjør om outputen til sannsynligheter, fører klassiske avstandsmål som MSE til et flatt og ugunstig tapslandskap (vanishing gradients). Kryssentropi måler i stedet den statistiske avstanden mellom to sannsynlighetsfordelinger, noe som gir en optimal gradient for læring.

ENTROPI OG BESLUTNINGSTRÆR

Samme informasjonsteoretiske idé dukker opp i **beslutningstrær**, men der kalles det **entropi**:

- **Entropi** $H(S)$ måler **urenhet** eller usikkerhet i et sett S :

$$H(S) = - \sum_{k=1}^K p_k \log_2(p_k)$$

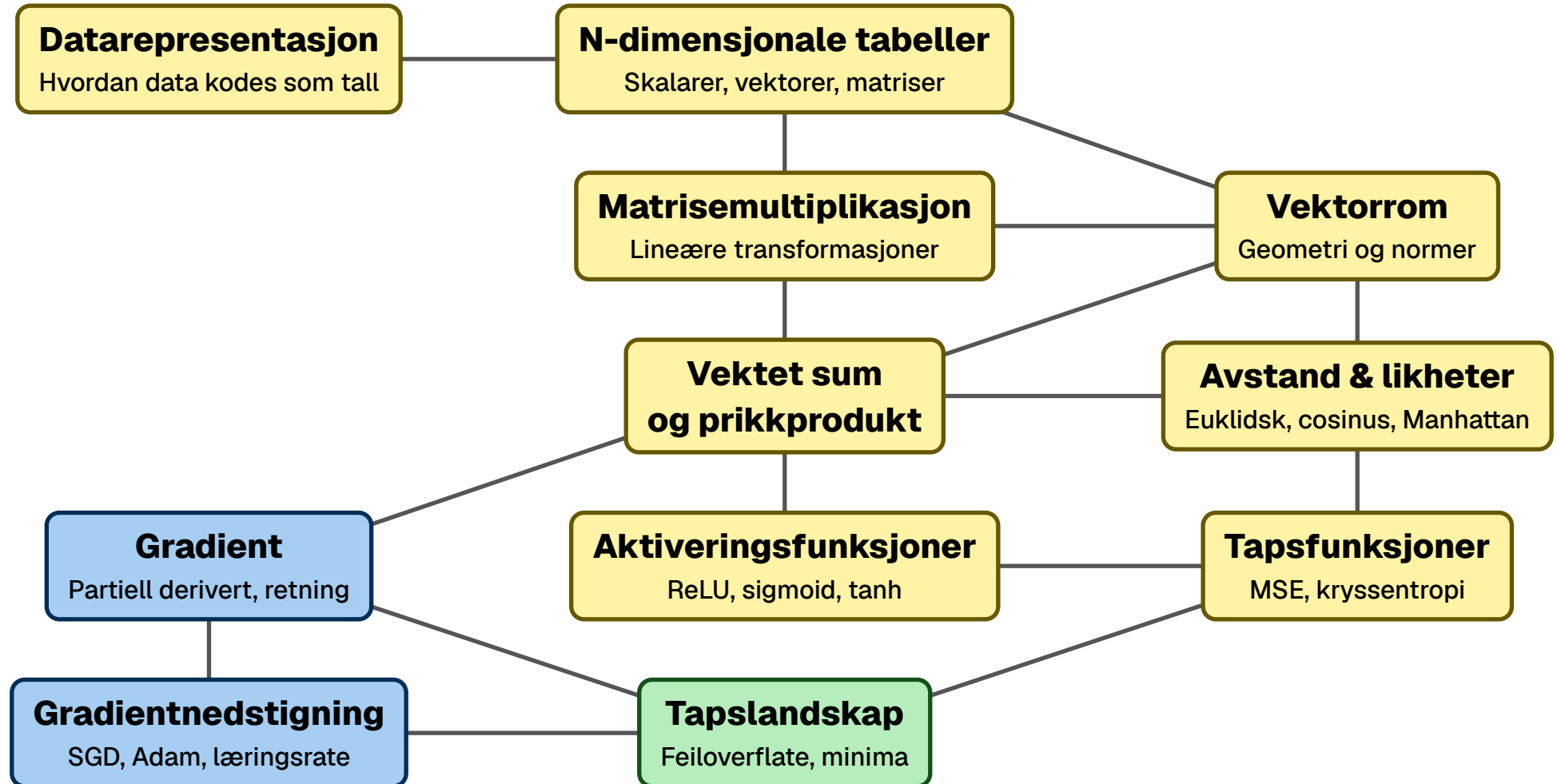
- p_k = andelen av klasse k i settet S
- $H = 0$: alle eksempler tilhører samme klasse — perfekt ren node.
- H er maksimal når alle klasser er like sannsynlige — maksimal forvirring.

Informasjonsgevinst ved splitting på feature A forteller oss **hvor mye** renere nodene blir:

$$\text{IG}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

Sammenheng: Kryssentropi måler den informasjonsteoretiske 'avstanden' (kjent som KL-divergens) mellom modellens predikerte fordeling og den sanne fordelingen y . Begge MSE, kryssentropi og entropi er ulike former for det samme spørsmålet: «*Hvor langt unna sannheten er vi?*»

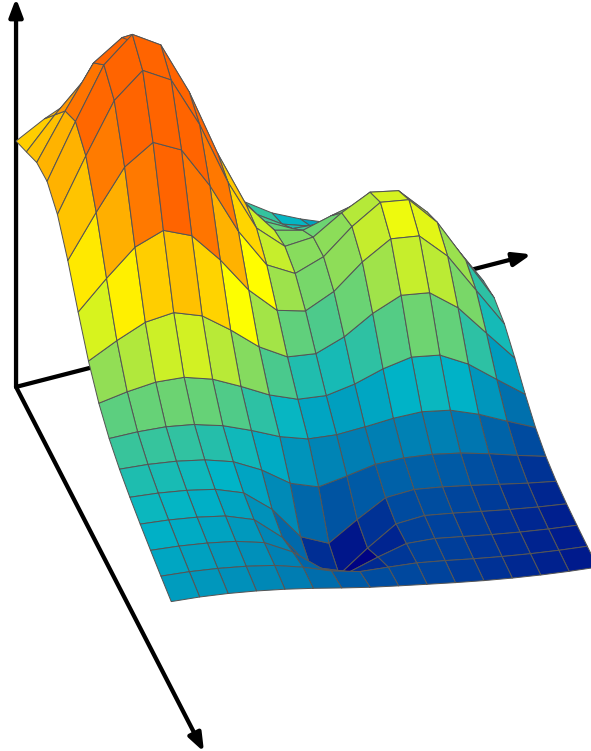
TAPSLANDSKAP



VISUALISER OPTIMERINGSPROBLEMER

Tapsfunksjonen gir oss ett tall for ett sett med vekter. Men modellen har **mange** vekter — og vi kan tenke på alle mulige kombinasjoner som et høydimensjonalt **landskap**.

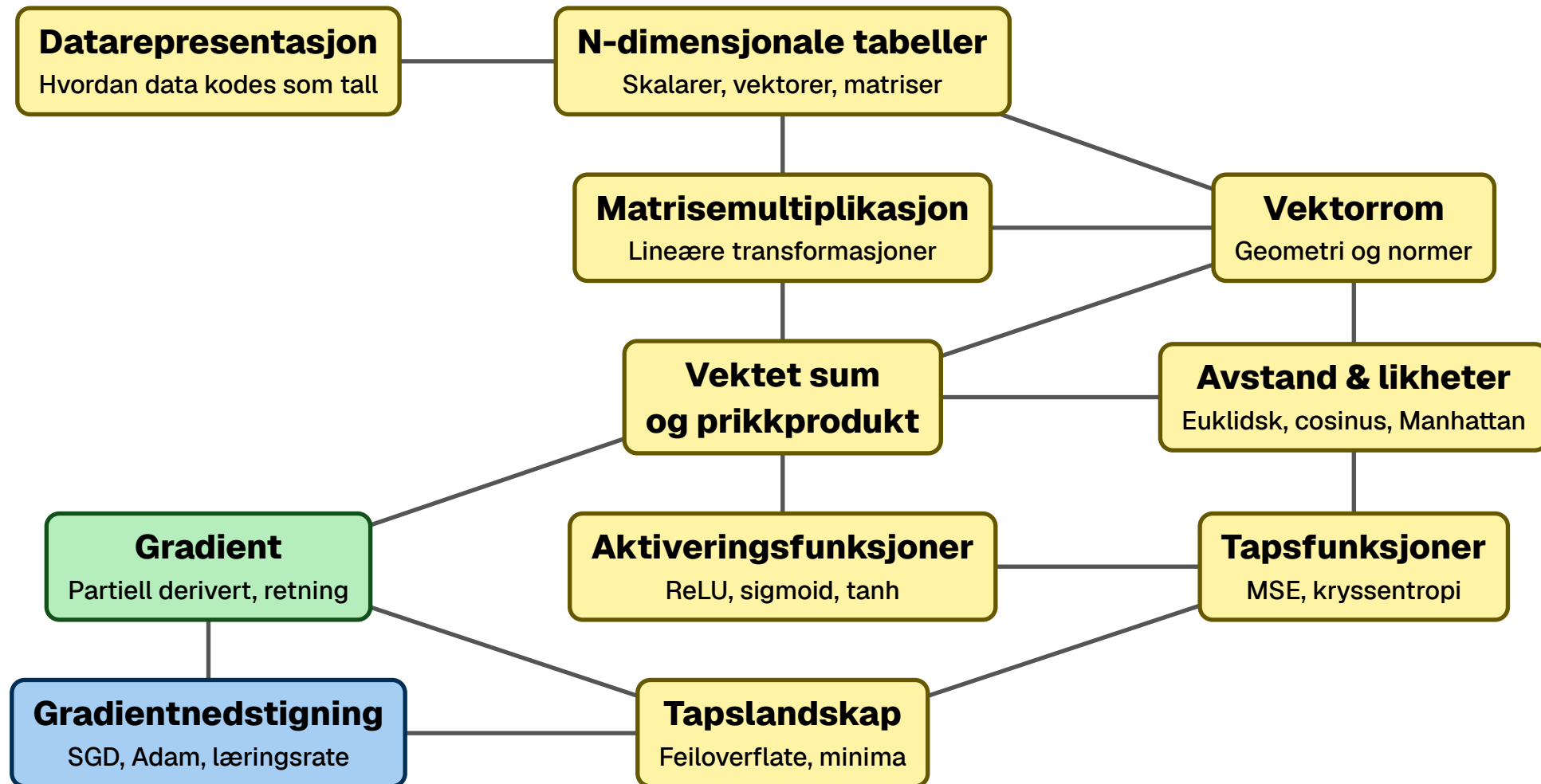
- Hvert punkt i landskapet tilsvarer ett sett med vekter $\mathbf{W}$.
- Høyden i punktet er tapet $L(\mathbf{W})$ — feilmålingen vår.
- **Daler** er regioner med lavt tap — gode modeller.
- **Fjell og topper** er regioner med høyt tap — dårlige modeller.
- Målet med trening er å **finne dalbunnen** — der tapet er lavest.



Hvilken retning skal vi gå for å komme nedover?

Oppgave: Hvor mange trekk (vekker) former dette landskapet?

GRADIENT



HVA ER EN GRADIENT?

Husk at vi ønsker å bevege oss **nedover** i tapslandskapet. Gradienten forteller oss hvilken vei **opp** er — og da går vi den andre veien.

- Gradienten $\nabla L(\mathbf{W})$ er en **vektor** som peker i retningen der tapet **øker raskest**.
- Den inneholder én partiell derivert per parameter:

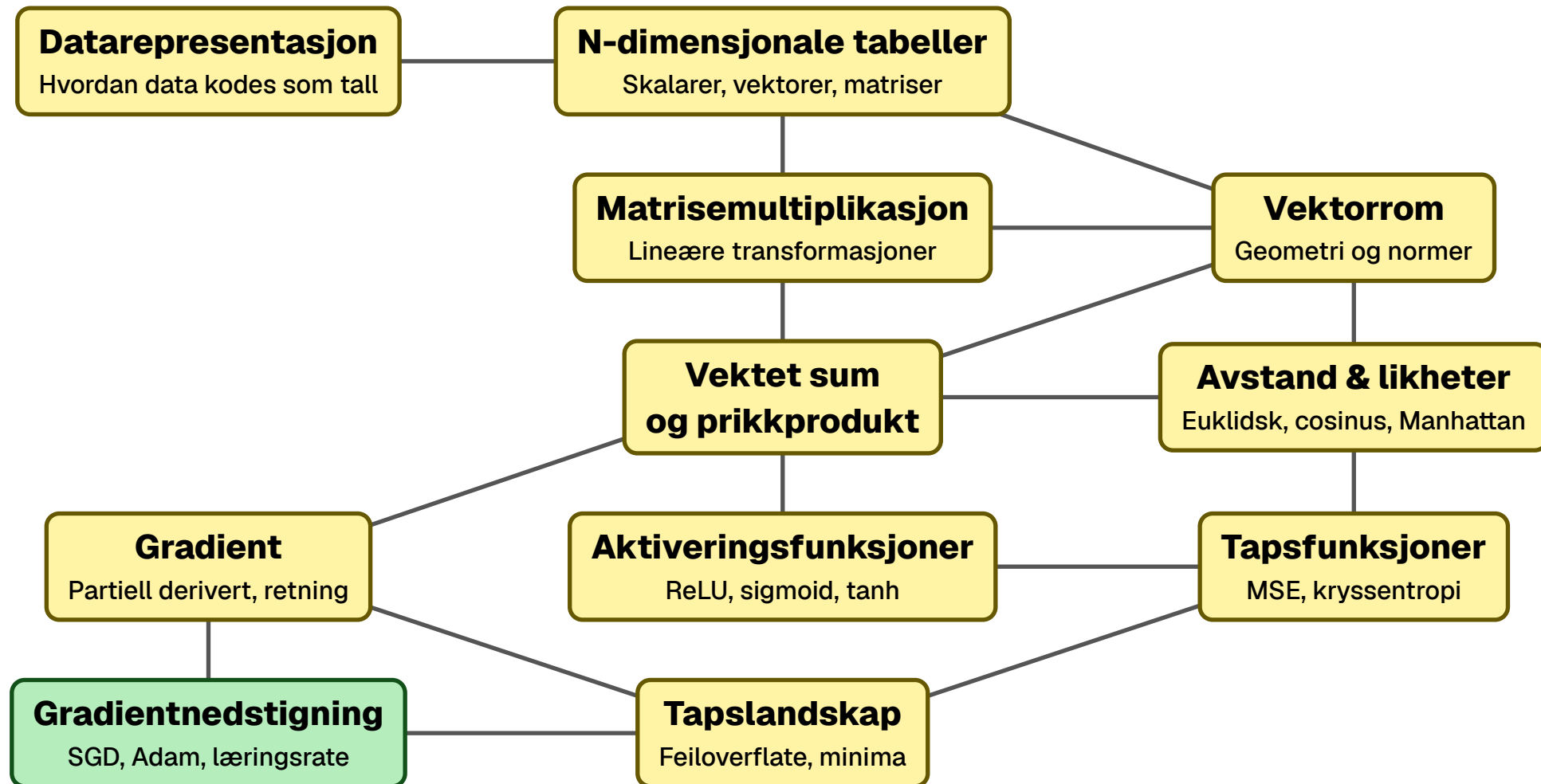
$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots \right)$$

- Størrelsen (normen) til gradienten forteller oss **hvor bratt** terrenget er akkurat nå.

Intuisjon: Tenk på gradienten som en kompassnål som alltid peker oppover i tapslandskapet. For å komme ned i dalen, går vi i **motsatt retning** av gradienten.

Kobling til det vi har lært: Gradienten beregnes via kjerneregelen (backpropagation), og hvert steg innebærer matrisemultiplikasjon og prikkprodukter — de samme operasjonene vi allerede kjenner fra foroverkjøringen. Matematikken er konsistent hele veien gjennom.

GRADIENTNEDSTIGNING



OPPDATERINGSREGELEN

Nå som vi vet hvilken retning vi skal gå, tar vi ett skritt i den retningen. Dette er **gradientnedstigning**.

Vi oppdaterer alle vektene ved å trekke fra gradienten, skalert med læringsraten:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L$$

- η (eta) er **læringsraten** — størrelsen på hvert steg i tapslandskapet.
- For liten η : treningen er sakte, vi kryper nedover og bruker lang tid.
- For stor η : vi «hopper over» dalbunnen og modellen divergerer.

Mini-batch gradientnedstigning: I praksis beregner vi gradienten på en liten **batch** av eksempler om gangen — ikke hele datasettet. Dette gjør treningen raskere og introduserer nok støy til at modellen unnslipper lokale minima.

Fra start til slutt: Data inn som vektor $\rightarrow$ matrisemultiplikasjon lag for lag $\rightarrow$ aktiveringsfunksjon $\rightarrow$ tapsfunksjon $\rightarrow$ gradient $\rightarrow$ oppdater vektorer $\rightarrow$ gjenta. Dette er treningsloopen i all moderne dyp læring.

OPPSUMMERING

Vi har nå gått gjennom hele den matematiske kjeden som gjør maskinlæring mulig:

- **N-dimensjonale matriser:** Alt data representeres som skalarer, vektorer, matriser og tensorer.
- **Datarepresentasjon:** Rå data — tekst, bilder, kategorier — konverteres til numeriske vektorer.
- **Vektorrom:** Vektorer bor i et geometrisk rom der avstand er veldefinert.
- **Avstand og likhet:** Euklidsk distanse og cosinus-likhet lar oss sammenligne datapunkter.
- **Prikkprodukt og vektet sum:** Kjerneoperasjonen — én score per datapunkt.
- **Matrisemultiplikasjon:** Mange prikkprodukter parallelt — transformerer data lag for lag.
- **Aktiveringsfunksjoner:** Introduserer ikke-linearitet slik at nettverket kan lære komplekse mønstre.
- **Tapsfunksjoner:** Måler geometrisk avstand mellom prediksjon og fasit.
- **Tapslandskap:** Alle mulige vekter danner et landskap — vi søker dalbunnen.
- **Gradient:** Peker oppover i landskapet — vi går motsatt vei.
- **Gradientnedstigning:** Iterativ oppdatering av vekter langs gradienten — selve læringsalgoritmen.

Den røde tråden: Hvert konsept bygger på det forrige. Geometri → algebra → kalkulus → optimering.
Maskinlæring er matematikk med data.

DISKUSJON